rsedit — Rapporto sui problemi del codice

Le schede dell'analisi della revisione `08a853d`, verificate sulla revisione `3d846f1`

Indice

1 Che cosa contiene questo rapporto 2
1.1 Le schede dell'interprete 2
1.2 Come leggere una scheda 2
1.3 Come è stato condotto l'esame 2
1.4 Riepilogo 3
2 Buffer, modifiche e annullamento 6
3 Tastiera, comandi e primitive dell'editor 7
4 Minibuffer 9
5 File e salvataggio 10
6 Avvio, configurazione e registro 12
7 Lavoro in background 14
8 Modi, sintassi e colorazione 15
9 Ricerca, sostituzione e liste di risultati 17
10 Comandi di shell 19
11 Disegno e terminale 20
12 Documentazione, commenti, codice morto, build e test 23
13 Appendici 25
13.1 Le schede risolte 25
13.2 Differenze da Emacs che non sono difetti 25
13.3 Parti esaminate e trovate solide 25

1 Che cosa contiene questo rapporto

Questo documento elenca i problemi trovati analizzando il codice di rsedit. È separato di proposito dal manuale di architettura, `architettura-editor.typ`. Il manuale descrive come il codice funziona e resta valido anche dopo una correzione; questo rapporto descrive dove il codice non fa ciò che dovrebbe. Quando un problema viene risolto, la sua scheda passa nell'appendice delle schede risolte, con il commit che lo ha risolto.

Ogni problema ha un identificativo stabile, formato dalla sigla dell'area e da un numero (`BUF-2`, `CMD-1`…), che si può citare in un commit o in una discussione; nel codice lo si cita solo in un `TODO(ID)`, e solo finché la scheda è aperta. Le schede sono generate da un'unica tabella di dati all'inizio del sorgente: per aggiungerne una, o per segnarne una come risolta, basta modificare quella tabella, e il riepilogo si aggiorna da solo.

1.1 Le schede dell'interprete

Le schede delle aree LET (il lettore) e INT (il valutatore e la libreria di base) riguardano l'interprete, che da `6d30567` è il crate esterno `risp`. Sono passate nel rapporto del suo repository, `doc/rapporto-problemi.typ`, con gli stessi identificativi. Le schede di qui che vi rimandano lo dicono nel testo, per esempio «INT-2, nel rapporto di risp». rsedit usa la revisione di risp fissata nel suo `Cargo.lock`, oggi `b2d000b`: una scheda che risp risolve dopo quella revisione resta un difetto di rsedit finché la dipendenza non si aggiorna.

1.2 Come leggere una scheda

Ogni scheda riporta:

- **Gravità**, secondo la scala qui sotto.
- **Dove**: i file e le funzioni, o i tipi, in cui sta il difetto. I percorsi senza prefisso sono relativi a `core/src/`; quelli che cominciano con `src/` riguardano la TUI e la GUI, quelli con `core/lisp/` i moduli Lisp.
- **Verifica**: «verificato» se il comportamento è stato riprodotto eseguendo il codice (il testo dice come, e su quale revisione), «dal codice» se la conclusione viene dalla sola lettura.
- **Cosa succede**, il meccanismo del difetto; **Come riprodurlo**, quando serve; **Effetto**, ciò che l'utente vede; **Correzione proposta**.

Tutte le schede dell'elenco sono aperte; le risolte sono nell'appendice.

Gravità	Significato
ALTA	L'editor termina, si blocca senza via d'uscita o perde dati; oppure un contenuto esterno può far eseguire azioni al terminale; oppure l'editor finisce in uno stato da cui si esce solo riavviando.
MEDIA	Un comportamento sbagliato o un rallentamento di secondi in un uso realistico, senza perdita di dati.
BASSA	Casi limite, incoerenze, codice morto, commenti e documentazione superati.

1.3 Come è stato condotto l'esame

Il codice della revisione `08a853d` è stato letto per intero: l'interprete (allora `core/src/lisp`, ora il crate `risp`), la libreria dell'editor (`core/src`), la TUI (`src/`) e i moduli Lisp (`core/lisp`). Ogni comportamento sospetto è stato riprodotto, dove possibile, con piccoli programmi esterni che usano `rsedit_core` come una libreria:

- un **esecutore di scenari** che valuta espressioni Lisp, preme tasti attraverso il percorso reale dei tasti, attende i lavori di background e stampa buffer, area messaggi ed evidenziazioni della fotografia dello schermo;

- un **fuzzer delle primitive**, che chiama ciascuna delle 628 funzioni registrate (tranne quelle che toccano file o processi esterni) con 807 combinazioni di argomenti anomali (zero, negativi, 10^{23}, infinito, NaN, stringhe vuote e non ASCII, liste puntate, vettori, mappe, lambda), una volta su un buffer vuoto e una su un buffer con testo multiriga, caratteri non ASCII, tab, una regione attiva e finestre divise.

I fuzzer del lettore e delle forme speciali, che hanno trovato le schede dell'interprete, sono descritti nel rapporto di risp. Le misure di tempo sono state prese con build di release su una macchina condivisa, quindi vanno lette come ordini di grandezza.

1.3.1 La verifica sulla revisione `3d846f1`

Ogni scheda è stata poi verificata sulla revisione `3d846f1`. Il codice che descrive è stato riletto, e la storia di git ha detto quale commit lo ha cambiato. Dove la scheda dice «verificato», la riproduzione è stata eseguita di nuovo, con lo stesso esecutore di scenari o con due editor veri in un terminale virtuale. Le misure di tempo e di memoria non sono state ripetute dove il codice misurato non è cambiato, e la scheda lo dice. Il campo **Dove** nomina ora file e funzioni, non righe.

Delle 51 schede di rsedit, 11 sono risolte. Sette lo sono in parte e restano aperte con il testo aggiornato a ciò che manca: BUF-1, CMD-1, CMD-3, CMD-6, MIN-3, FIL-5 e DOC-1. La verifica ne ha trovate quattro nuove: AVV-6, BKG-2, RIC-7 e DOC-5.

1.4 Riepilogo

> Sono aperti **44 problemi**: **4 di gravità alta**, 17 di gravità media e 23 di gravità bassa; 27 sono stati riprodotti eseguendo il codice. Altri 11 sono stati risolti dopo l'analisi, e sono nell'appendice.

Dei quattro problemi di gravità alta, due riguardano il terminale: un file può mandargli sequenze di controllo (TUI-1), e un panic o un errore lo lasciano in raw mode (TUI-2). Gli altri due fanno perdere lavoro senza dirlo: un salvataggio fallito non viene segnalato (FIL-1), e due prompt aperti insieme perdono il testo del primo e lasciano una finestra che non si chiude (MIN-1). Delle tre correzioni trasversali che l'analisi proponeva, una c'è: nel WorkManager un panic ritira solo il lavoro che l'ha causato (BKG-1, risolta). Restano da fare la guardia che ripristini il terminale (TUI-2) e un limite alle quantità guidate dall'utente, nelle primitive dell'editor (CMD-3) come nell'interprete (INT-2 e INT-13, nel rapporto di risp).

La tabella seguente elenca i problemi aperti, dal più grave; l'identificativo rimanda alla scheda.

ID	Problema	Gravità	Verifica
MIN-1	Prompt annidati: testo perso e finestra flottante fantasma	ALTA	verificato
FIL-1	`save-buffer` fallito: solo una riga nel registro	ALTA	verificato
TUI-1	Caratteri di controllo stampati così come sono: iniezione di sequenze nel terminale	ALTA	verificato
TUI-2	Nessun ripristino del terminale su panic o errore	ALTA	dal codice
BUF-1	`undo` / `redo` scavalcano le porte: sola lettura, overlay, testo virtuale	MEDIA	verificato
BUF-2	`GapBuffer::cursor_pos` lineare nella posizione	MEDIA	verificato
CMD-1	Tasti non ASCII: non digitabili con la configurazione predefinita	MEDIA	verificato
CMD-2	`KeyCode` senza Delete/Home/End/PgUp/PgDn/F1…	MEDIA	dal codice
CMD-3	Larghezze e conteggi senza limite nelle primitive Rust: panic e abort	MEDIA	verificato
MIN-2	`C-g` nel minibuffer non chiude il prompt	MEDIA	verificato
FIL-2	Scrittura non atomica dei file	MEDIA	dal codice
FIL-3	Aprire un file non UTF-8 fallisce in silenzio	MEDIA	verificato
AVV-2	`eval_file`: errori di sintassi silenziosi; un commento finale senza a capo annulla il file	MEDIA	verificato

ID	Problema	Gravità	Verifica
SIN-1	Regioni che corrispondono a «»: ciclo infinito o memoria esaurita nell'evidenziatore	MEDIA	verificato
SIN-2	Colorazione quadratica sulle righe lunghe	MEDIA	verificato
SIN-6	`C-M-f` a metà di un simbolo: un'espressione di troppo	MEDIA	verificato
RIC-1	`replace-regexp` con match vuoti: tutto nello stesso punto	MEDIA	verificato
RIC-2	`C-M-r`: tempo quadratico per tasto	MEDIA	verificato
TUI-3	Tab non espansi: cursore, colori e riempimento sfasati	MEDIA	verificato
TUI-4	CJK ed emoji contati come una colonna	MEDIA	dal codice
TUI-5	`Color::parse`: panic su colori non ASCII	MEDIA	verificato
BUF-3	`GapBuffer::clone` è `todo!()`; `find_forward` underflow	BASSA	dal codice
CMD-4	`parse_key`: un solo prefisso; `define-key` fallisce in silenzio	BASSA	verificato
CMD-5	`C-u 3 x` inserisce una sola `x`	BASSA	verificato
CMD-6	`kill-word`, `delete-char` e simili: un conteggio negativo vale 0	BASSA	dal codice
MIN-3	`set_minibuffer_content` scavalca le porte e lascia il testo dopo il punto	BASSA	verificato
FIL-4	Corsa tra copia del testo e azzeramento di «modificato»	BASSA	dal codice
FIL-5	Autosave: versione segnata anche se la scrittura fallisce	BASSA	dal codice
AVV-5	`init.lisp` predefinito scritto solo alla prima esecuzione	BASSA	dal codice
AVV-6	Due editor insieme: il secondo tronca il registro del primo, e le righe si sovrappongono	BASSA	verificato
BKG-2	Un panic ritira il lavoro per sempre: colorazione o salvataggio automatico fermi senza avviso	BASSA	dal codice
SIN-4	`make-mode` ricrea il modo, perdendo legami e hook	BASSA	dal codice
SIN-5	Facce: registro globale senza limite, indice `u16`	BASSA	dal codice
RIC-3	Una sostituzione = un passo di `undo`	BASSA	verificato
RIC-4	Match oltre il limite scartato; colonna ricalcolata per ogni match	BASSA	verificato
RIC-5	Risultati per numero di riga, non per marcatore	BASSA	verificato
RIC-7	Liste di occur e di grep in due strisce separate	BASSA	verificato
SHL-4	`manpage`: nome interpolato senza quoting nella shell	BASSA	verificato
TUI-6	Finestre flottanti senza sintassi e overlay	BASSA	dal codice
TUI-7	`%p` = `All` solo con una riga	BASSA	dal codice
TUI-8	Niente schermo alternativo: all'uscita lo schermo viene cancellato	BASSA	dal codice
DOC-1	Commenti e docstring superati	BASSA	dal codice
DOC-2	`local_keymap` mai usato; `Debug` di `EditorState` è `todo!()`	BASSA	dal codice
DOC-5	`free_buffer_name` mai usato; due `if` da unire; 41 blocchi da formattare	BASSA	verificato

Area	Descrizione	Alta	Media	Bassa	Risolte
BUF	Buffer, modifiche e annullamento	0	2	1	0
CMD	Tastiera, comandi e primitive dell'editor	0	3	3	0
MIN	Minibuffer	1	1	1	0
FIL	File e salvataggio	1	2	2	0
AVV	Avvio, configurazione e registro	0	1	2	3
BKG	Lavoro in background	0	0	1	1
SIN	Modi, sintassi e colorazione	0	3	2	1
RIC	Ricerca, sostituzione e liste di risultati	0	2	4	1
SHL	Comandi di shell	0	0	1	3
TUI	Disegno e terminale	2	3	3	0
DOC	Documentazione, commenti, codice morto, build e test	0	0	3	2

Tabella 1: I problemi aperti per area e gravità, e quelli risolti

2 Buffer, modifiche e annullamento

BUF-1 — `undo` e `redo` modificano il testo senza passare dalle due porte delle modifiche

Gravità	MEDIA
Dove	`primitives/edits.rs` (le primitive `undo` e `redo`), `buffer/undo.rs` (`UndoHistory::undo`, `UndoHistory::redo`, `apply_inverse`), `editor/replace.rs` (`EditorState::replace_back`)
Verifica	Riprodotto sulla revisione `3d846f1`: dopo `(insert "abc")`, `(undo-boundary)` e `(set-buffer-read-only t)`, `(undo)` svuota il buffer senza messaggi; un overlay su `world` in `XXhello world` resta agli offset 8–13 dopo che `undo` ha tolto `XX`. La colorazione e le cache, che l'analisi aveva trovato vecchie dopo `C-/`, ora si aggiornano: lo provano i test `undo_and_redo_recolour_what_they_change` e `an_undo_does_not_leave_a_stale_answer_behind`.

Cosa succede. Ogni modifica dovrebbe passare da `insert_text` o `delete_range`. Sono loro che controllano la sola lettura, incrementano `version`, invalidano le cache di colorazione e di scansione, disattivano il mark e spostano overlay e testo virtuale. `undo` e `redo` applicano invece le modifiche inverse direttamente sul testo (`apply_inverse`). Da `9956bed` chiamano poi `changed`, che incrementa la versione, invalida le cache dalla prima riga toccata ed emette `buffer-changed`; il resto delle due porte resta saltato.

Come riprodurlo.

```
(insert "abc") (undo-boundary)
(set-buffer-read-only t)
(undo)                      ; il buffer si svuota, senza «Buffer is read-only»
```

Effetto. (1) `undo` modifica i buffer in sola lettura. (2) Overlay e testo virtuale restano agli offset del testo vecchio, quindi le evidenziazioni cadono sulle parole sbagliate.

Correzione proposta. Far applicare a `UndoHistory` i suoi passi attraverso le due porte (o una terza porta che faccia le stesse cose senza registrare l'annullamento), e rifiutare `undo` in un buffer in sola lettura.

BUF-2 — `cursor_pos()` costa O(posizione): ogni fotogramma rilegge il buffer fino al punto

Gravità	MEDIA
Dove	`buffer/gap_buffer.rs` (`cursor_pos`); la chiamano a ogni fotogramma `ui/windows.rs` (la composizione delle finestre ed `expand_mode_line`) ed `editor/frame.rs` (`EditorState::snapshot`, per una finestra flottante)
Verifica	Misurato nell'analisi di `08a853d` su un buffer di 8 MB: `snapshot` impiegava 7,9 ms con il punto alla fine e 0,03 ms con il punto all'inizio. `buffer/gap_buffer.rs` non è cambiato da allora.

Cosa succede. Per trovare riga e colonna del punto `cursor_pos` scorre tutti i caratteri dall'inizio del buffer, mentre `cursor_1d_to_2d`, accanto, cerca già la riga nell'indice dei ritorni a capo. Ogni fotogramma la chiama almeno due volte per finestra (posizione del cursore e riga di stato), e circa trenta altri punti la usano.

Effetto. In un file grande ogni tasto costa millisecondi in più, e il costo cresce con la distanza del punto dall'inizio.

Correzione proposta. Ricerca binaria della riga nell'indice dei ritorni a capo, poi un conteggio dei soli caratteri della riga.

BUF-3 — `GapBuffer::clone` è `todo!()` e `find_forward` va in underflow su un buffer vuoto

Gravità	BASSA
Dove	`buffer/gap_buffer.rs` (`impl Clone for GapBuffer`, `find_forward`), `buffer/buffer_trait.rs` (`BufferTrait`, che richiede `Clone`)
Verifica	Dalla lettura del codice; oggi nessun percorso li raggiunge (`find_forward` è usata solo nei test).

Cosa succede. `BufferTrait` richiede `Clone`, ma l'implementazione per `GapBuffer` è un `todo!()`: il primo che clonerà un buffer manderà l'editor in panic. `find_forward` calcola `self.len() - 1`, che su un testo vuoto va in underflow (panic in debug) e poi arriva a un `unreachable!()`.

Correzione proposta. Derivare `Clone`, oppure togliere il vincolo dal trait; in `find_forward` usare `checked_sub`.

3 Tastiera, comandi e primitive dell'editor

CMD-1 — I caratteri non ASCII non si possono digitare

Gravità	MEDIA
Dove	`input.rs` (`fill_default_keymaps`, che lega a `self-insert` solo l'ASCII stampabile, e `Keymap::binding`, con il ripiego per i caratteri), `editor/keys.rs` (`EditorState::handle_key_event`)
Verifica	Riprodotto sulla revisione `3d846f1`: premendo `è` l'area messaggi mostra «è is undefined»; `(define-key nil "è" 'forward-char)` ora restituisce `t`.

Cosa succede. La mappa globale lega a `self-insert` un tasto per ogni carattere da spazio a tilde; ogni altro carattere arriva alla risoluzione dei tasti senza un comando. Da `0a87fee` `parse_key` conta i caratteri e non i byte, quindi un carattere non ASCII si può legare, e una mappa può dare un comando a tutti i caratteri senza un legame, con `(define-key MAPPA "<char>" COMANDO)`. Ma lo fanno solo le mappe di vim: la mappa globale non ne ha uno.
Effetto. Con la configurazione predefinita l'editor è inutilizzabile per scrivere in italiano, francese, tedesco o in qualunque lingua con lettere accentate, e per i simboli tipografici (`€`, `°`, `«»`).
Correzione proposta. Dare alla mappa globale un comando per i caratteri, che inserisca il carattere premuto, come fa `self-insert-command` di Emacs.

CMD-2 — Mancano Canc, Inizio, Fine, PagSu, PagGiù e i tasti funzione

Gravità	MEDIA
Dove	`input.rs` (`KeyCode`), `src/tui.rs` (`translate_key`, che scarta tutto il resto), `primitives/mod.rs` (`parse_key`, senza un nome per questi tasti); nella GUI, `src/gui.rs` (`EditorApp::native_key`, `special_key`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(define-key nil "<delete>" 'delete-char)`, e lo stesso con `"<home>"` e `"<f1>"`, danno `nil` e «Invalid key sequence» nel registro, dove la stessa riga per `<delete>` c'è già dall'avvio. Il resto dalla lettura del codice: l'enum ha solo `Char`, `Backspace`, `Enter`, `Esc`, `Tab`, le quattro frecce e un segnaposto `None`.

Cosa succede. I tasti non rappresentati vengono scartati dalla TUI con una riga nel registro («no translation for the key»), e `parse_key` non ha un nome per loro: il modulo `electric-pair` lega `<delete>` e fallisce a ogni avvio. La GUI (`src/gui.rs`) traduce da sé Inizio, Fine, Canc, PagSu e PagGiù in `C-a`, `C-e`, `C-d`, `M-v` e `C-v`, o nei tasti di vim nello stato normale: lì quei tasti funzionano, ma non si possono legare. I tasti funzione mancano in tutte e due. Le pagine `editing` e `keymaps` lo dicono tra i limiti.
Effetto. Tasti che ogni utente si aspetta (Canc in particolare) non fanno nulla nel terminale, e non si possono legare né lì né nella GUI.
Correzione proposta. Estendere `KeyCode` e `translate_key`, aggiungere i nomi (`<delete>`, `<home>`, `<f1>`…) a `parse_key`, e togliere dalla GUI le traduzioni fisse in favore di legami nelle mappe.

CMD-3 — Argomenti numerici enormi: allocazioni che terminano il processo e panic

Gravità	MEDIA
Dove	`primitives/edits.rs` (`indent-line-to`, `repeat_count`), `primitives/macros.rs` (`kmacro-insert-counter`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(kmacro-insert-counter 70000)` va in panic con «Formatting argument out of range»; `(forward-word 300000000)` invece si ferma subito alla fine del buffer. `indent-line-to` non è cambiata dall'analisi, che l'aveva trovata con il fuzzer delle primitive.

Cosa succede. Alcune primitive scritte in Rust usano un numero dell'utente come quantità da allocare, senza un limite e senza carburante. `indent-line-to` crea una stringa di spazi lunga quanto la colonna richiesta. `kmacro-insert-counter` passa la larghezza a `format!`, che oltre 65 535 va in panic. Da `cf0d8ac` i comandi sulle parole si fermano invece al bordo del buffer, e non ripetono più a vuoto il loro passo.
Come riprodurlo.

```
(indent-line-to 300000000000)     ; abort: memory allocation failed
(kmacro-insert-counter 65536)     ; panic: Formatting argument out of range
```

Effetto. Un panic nel thread dei comandi chiude l'editor e lascia il terminale in raw mode (TUI-2); un'allocazione rifiutata termina il processo senza nemmeno un panic.

Correzione proposta. Limitare larghezze e conteggi (per esempio a un milione) con un errore `args-out-of-range`, a partire da `repeat_count`; usare `try_reserve` per le allocazioni guidate dall'utente.

CMD-4 — Le combinazioni di modificatori in `define-key` sono limitate e un errore va solo nel registro

Gravità	BASSA
Dove	`primitives/mod.rs` (`parse_key`), `primitives/general.rs` (`define-key`)
Verifica	Riprodotto sulla revisione `3d846f1`: `"M-C-x"` e `"C-S-a"` danno `nil`, `"C-M-x"` dà `t`.

Cosa succede. `parse_key` riconosce un solo prefisso tra `C-M-`, `C-`, `M-` e `S-`, quindi `M-C-x` (stesso tasto di `C-M-x`) e le combinazioni con Shift non si possono scrivere. Una sequenza non valida o un modo inesistente fanno restituire `nil` a `define-key`, con il motivo scritto solo nel registro. La docstring di `define-key` e la pagina `keymaps` ora dicono quali combinazioni non sono tasti.

Correzione proposta. Analizzare i prefissi in un ciclo, in qualsiasi ordine, e segnalare l'errore con un errore Lisp.

CMD-5 — `self-insert` ignora l'argomento prefisso

Gravità	BASSA
Dove	`input.rs` (`fill_default_keymaps`: il legame è `(self-insert "x")`, una forma che fornisce già i suoi argomenti), `editor/keys.rs` (`EditorState::handle_key_event`), `primitives/edits.rs` (`self-insert`)
Verifica	Riprodotto sulla revisione `3d846f1`: `C-u 3 x` inserisce una `x`.

Cosa succede. Una forma che fornisce già i suoi argomenti non riceve l'argomento prefisso, e `self-insert` non lo legge. La pagina `editing` ora lo dice tra i conteggi; in Emacs le `x` sono tre.

Correzione proposta. Legare i caratteri a un comando che legge il tasto premuto (come `self-insert-command` di Emacs), oppure passare il prefisso alle forme di `self-insert`.

CMD-6 — Un conteggio negativo non inverte i comandi che cancellano

Gravità	BASSA
Dove	`primitives/edits.rs` (`repeat_count`, e i comandi che lo usano: `delete-char`, `kill-word`, `backward-kill-word`, `kill-sexp`, `backward-kill-sexp`)
Verifica	Riprodotto sulla revisione `3d846f1`: in `foo bar`, `(kill-word -1)` dalla fine e `(delete-char -1)` da metà non cambiano il testo.

Cosa succede. `repeat_count` trasforma un conteggio negativo in 0, quindi i comandi che lo usano non fanno nulla, mentre in Emacs agiscono all'indietro. Due parti della scheda sono risolte: da `4506432` una risposta non numerica a una domanda numerica viene chiesta di nuovo, e da `cf0d8ac` gli spostamenti (caratteri, parole, righe, paragrafi, espressioni) usano `movement_count` e con un conteggio negativo vanno all'indietro.

Correzione proposta. Usare `movement_count` anche nei comandi che cancellano, cancellando all'indietro con un conteggio negativo.

4 Minibuffer

MIN-1 — Due domande aperte insieme condividono `*Minibuffer*` e lasciano una finestra fantasma

Gravità ALTA
Dove `primitives/minibuffer.rs` (`default-minibuffer-prompt`, che usa sempre il nome `*Minibuffer*`), `editor/windows.rs` (`EditorState::open_floating_window`), `editor/buffers.rs` (`EditorState::close_buffer`, che chiude la **prima** finestra flottante che mostra il buffer)
Verifica Riprodotto sulla revisione `3d846f1` con la sequenza `M-x one`, `M-x two`, Escape: con i due prompt aperti ci sono due finestre flottanti su `*Minibuffer*` e `one` è sparito; dopo Escape il fuoco è su `*scratch*` e una finestra flottante su `*Minibuffer*` resta aperta.

Cosa succede. Ogni prompt crea il buffer `*Minibuffer*` e una finestra flottante che lo mostra. Un secondo prompt aperto mentre il primo è ancora aperto (un `M-x` dentro `M-x`, o un comando che fa una domanda mentre un prompt è aperto) riusa lo stesso buffer, quindi il testo del primo si perde e le due finestre mostrano lo stesso testo. Escape chiude il buffer e la **prima** finestra trovata, cioè quella del prompt esterno. Resta così a schermo la finestra del prompt interno, senza un buffer valido, e il fuoco torna a `*scratch*`. Il prompt successivo riusa il nome, e la finestra fantasma ne mostra il testo.
Effetto. Una finestra che non si può più chiudere copre parte dello schermo fino al riavvio, e la risposta che l'utente stava scrivendo nel prompt esterno è persa.
Correzione proposta. Nomi di buffer distinti per prompt (`*Minibuffer-1*`…) e una pila di prompt, oppure rifiutare un secondo prompt finché il primo è aperto, come fa Emacs senza `enable-recursive-minibuffers`. In `close_buffer`, chiudere **tutte** le finestre flottanti che mostrano il buffer.

MIN-2 — `C-g` dentro il minibuffer non annulla la domanda

Gravità MEDIA
Dove `core/lisp/common-keymaps.lisp` (`C-g` ⟶ `keyboard-quit`, anche nel minibuffer), `feature/minibuffer.rs` (`install_minibuffer`: la mappa di `minibuffer-mode` non lega `C-g`)
Verifica Riprodotto sulla revisione `3d846f1`: `M-x` poi `C-g` scrive «Quit» e il prompt resta aperto.

Cosa succede. `keyboard-quit` abbandona una sequenza di tasti a metà e azzera l'argomento prefisso, ma non sa nulla dei prompt aperti. La pagina `minibuffer` ora lo dice: Escape è la via d'uscita.
Effetto. Chi viene da Emacs preme `C-g` per uscire da una domanda e non ottiene nulla; il prompt resta aperto e cattura i tasti successivi.
Correzione proposta. Legare `C-g` a `minibuffer-cancel` in `minibuffer-mode`.

MIN-3 — Il contenuto del minibuffer viene sostituito senza passare dalle porte delle modifiche, e solo fino al punto

Gravità BASSA
Dove `feature/minibuffer.rs` (`set_minibuffer_content`), chiamata da `primitives/minibuffer.rs` (`minibuffer_complete`, `minibuffer_choose_completion`, `step_history`)
Verifica Riprodotto sulla revisione `3d846f1`: dopo un `M-x forward-char`, un nuovo `M-x`, `abc`, `C-a` e `M-p` lasciano nel minibuffer `forward-charabc`. Il resto dalla lettura del codice.

Cosa succede. La funzione che scrive la risposta proposta (completamento, cronologia) cancella il testo prima del punto e inserisce quello nuovo direttamente nel testo: quello che sta dopo il punto resta. Da `c850aee` chiama poi `changed`, quindi versione e cache sono a posto; il resto delle porte resta saltato: nulla per l'annullamento, nessun controllo della sola lettura, overlay e testo virtuale non spostati (BUF-1).
Effetto. Con il punto a metà della risposta, la cronologia non la sostituisce ma ci si mette davanti, e la risposta che parte è sbagliata.
Correzione proposta. Sostituire tutta la risposta, con `delete_range` e `insert_text`.

5 File e salvataggio

FIL-1 — Un salvataggio fallito con `C-x C-s` non viene segnalato

Gravità ALTA
Dove `primitives/io.rs` (`save-buffer`, `save-buffer--overwrite`, e `write_buffer`, che usano anche `save-some-buffers` e l'uscita); per confronto `write-file`
Verifica Riprodotto sulla revisione `3d846f1`: `save-buffer` in un buffer senza file lascia l'area messaggi invariata e scrive «No file associated with this buffer» solo nel registro. Il caso dell'analisi, un percorso in una directory inesistente, passa dallo stesso ramo, che non è cambiato.

Cosa succede. L'errore di `std::fs::write` viene passato a `log_diagnostic`, che scrive nel registro e non nell'area messaggi. `write-file`, al contrario, lo mostra. La docstring di `save-buffer` ora dice che il motivo di un fallimento va nel registro.
Effetto. Chi salva su un disco pieno, su un file senza permessi di scrittura o su un percorso che non esiste più crede di aver salvato. L'unico avviso arriva all'uscita, con la domanda «… is unsaved. Quit anyway?», e solo se il buffer è ancora segnato come modificato.
Correzione proposta. Mostrare nell'area messaggi sia «Wrote PERCORSO» sia «Cannot save PERCORSO: motivo», come fa `write-file`.

FIL-2 — I file vengono scritti sul posto: un errore a metà li lascia troncati

Gravità MEDIA
Dove `primitives/io.rs` (`save-buffer`, `write-file`, `save-buffer--overwrite`, `write_buffer`) e `modes/autosave.rs` (`EditorState::write_auto_save`): tutti con `std::fs::write`
Verifica Dalla lettura del codice.

Cosa succede. `std::fs::write` tronca il file e poi scrive. Se la scrittura si interrompe (disco pieno, processo terminato, alimentazione), sul disco resta un file troncato o vuoto, e la versione precedente è già persa.
Effetto. Possibile perdita del contenuto del file proprio quando qualcosa va storto.
Correzione proposta. Scrivere in un file temporaneo nella stessa directory, fare `sync_all` e poi `rename` sull'originale, conservando i permessi.

FIL-3 — Un file non UTF-8 non si apre e non viene detto nulla

Gravità MEDIA
Dove `editor/buffers.rs` (`EditorState::new_buffer`: `read_to_string`, errore solo nel registro), `primitives/io.rs` (`find-file`)
Verifica Riprodotto sulla revisione `3d846f1` con un file Latin-1: area messaggi e buffer corrente invariati, `find-file` restituisce `nil`, e il registro dice «Error reading file: stream did not contain valid UTF-8».

Cosa succede. `std::fs::read_to_string` rifiuta qualsiasi byte non UTF-8; l'errore finisce nel registro e il comando non fa nulla di visibile. La docstring di `find-file` ora dice che un file che non si legge dà `nil`.
Effetto. File in Latin-1, file binari e file con un solo byte sbagliato sembrano non aprirsi «per nessun motivo».
Correzione proposta. Un messaggio nell'area messaggi; in prospettiva un'apertura con `String::from_utf8_lossy` in un buffer in sola lettura, o una decodifica Latin-1 di ripiego.

FIL-4 — Una modifica fatta durante il salvataggio può restare non salvata ma non segnata

Gravità BASSA
Dove `primitives/io.rs` (`save-buffer`, `save-buffer--overwrite`, `write_buffer`; `write-file` segue lo stesso schema)
Verifica Dalla lettura del codice.

Cosa succede. Il testo viene copiato sotto il lock, poi scritto senza lock, poi `is_modified` viene azzerato con un secondo lock, senza guardare `version`. Una modifica arrivata nel frattempo (da un lavoro Lisp in background o da un callback) resta nel buffer ma il buffer risulta salvato. Il salvataggio automatico non ha il problema: ricorda la versione che ha copiato.

Correzione proposta. Ricordare la `version` copiata e azzerare `is_modified` solo se è ancora quella.

FIL-5 — Il salvataggio automatico segna come salvato anche un tentativo fallito

Gravità	BASSA
Dove	`modes/autosave.rs` (`EditorState::write_auto_save`: `auto_saved_at` impostato anche dopo un errore)
Verifica	Dalla lettura del codice: il commento lo dice, «Recorded whether or not the write worked».

Cosa succede. Se la scrittura della copia fallisce per un motivo passeggero, la versione viene comunque registrata, e non si riprova finché il buffer non cambia. La prima metà della scheda è risolta da `1aa5cf7`: `auto_save_for` guarda file, sola lettura, modifica e versione già salvata prima di copiare il testo, e ogni giro guarda tutti i buffer e ne scrive al più quattro.
Correzione proposta. Registrare la versione solo dopo una scrittura riuscita.

6 Avvio, configurazione e registro

AVV-2 — Gli errori di sintassi nei file Lisp caricati vengono ignorati in silenzio

Gravità	MEDIA
Dove	`editor/boot.rs` (`EditorState::eval_file`: contenuto avvolto in `(progn …)`, errore del lettore → `Ok(nil)`); la usano `create_global_env`, per `init.lisp`, e la primitiva `eval-file`
Verifica	Riprodotto sulla revisione `3d846f1` con `eval-file`: un file con una parentesi non chiusa e un file la cui ultima riga è un commento senza a capo restituiscono `nil` senza messaggi, e nessuna delle loro forme viene valutata.

Cosa succede. Il file viene letto come un'unica forma `(progn CONTENUTO)`. Una parentesi in più chiude la `progn` prima del tempo e il resto non viene mai letto. Una parentesi mancante, o un commento sull'ultima riga senza a capo (la parentesi aggiunta finisce **dentro** il commento), lascia la `progn` aperta. In entrambi i casi il lettore fallisce e `eval_file` restituisce `nil` senza dire nulla.

Come riprodurlo.

```
init.lisp:
  (setq my-config-loaded 42)
  ;; fine della configurazione      ← senza a capo finale
→ dopo l'avvio my-config-loaded non è definita, e nessun messaggio
```

Effetto. L'utente vede l'editor partire senza la sua configurazione, o con metà della configurazione, e non c'è nulla che indichi perché.

Correzione proposta. Leggere le forme una alla volta e valutarle in sequenza, segnalando nell'area messaggi la riga del primo errore di sintassi. Come minimo, `format!("(progn\n{}\n)")`.

AVV-5 — La configurazione predefinita viene scritta solo se manca

Gravità	BASSA
Dove	`editor/boot.rs` (`DEFAULT_INIT_LISP`, e in `create_global_env` il blocco che scrive il file solo se manca), `paths.rs` (`app_dir`, per la directory della configurazione)
Verifica	Dalla lettura del codice.

Cosa succede. Il file `init.lisp` predefinito carica i moduli e imposta i legami. Una volta scritto non viene più toccato, quindi chi lo ha già non riceve i moduli aggiunti nelle versioni successive: dall'analisi se ne sono aggiunti quattro, `python-mode`, `toml-mode`, `bash-mode` e `vim`, che un `init.lisp` scritto prima non carica.

Correzione proposta. Separare la parte predefinita, caricata sempre dal codice o da un file di sistema, dalla parte personale, che è l'unica scritta nella directory dell'utente.

AVV-6 — Due editor aperti insieme scrivono lo stesso `rsedit.log` e si cancellano le righe a vicenda

Gravità	BASSA
Dove	`editor/log.rs` (`EditorState::enable_log_file`, con `File::create`), `managers/log.rs` (`Log::flush`, che scrive dalla posizione a cui era arrivato), `src/main.rs` (`main`, con un nome fisso nella directory della cache)
Verifica	Riprodotto sulla revisione `3d846f1` con due editor veri e la stessa directory di cache: all'avvio il secondo tronca il file e lo riscrive da capo; poi una riga del secondo («no translation for the key F(5)») e una del primo («Keymap not bound ») finiscono nello stesso punto, e il file dice «Keymap not bound » seguito dal resto dell'altra, «key F(5)».

Cosa succede. Il registro è sempre `rsedit.log` nella directory della cache, aperto con `File::create`, che tronca il file, e senza `O_APPEND`. Un secondo editor tronca quindi il file del primo e ci scrive dall'inizio; il primo continua a scrivere dalla posizione a cui era arrivato, sopra le righe dell'altro. La pagina `diagnostics` dice che il file ricomincia a ogni avvio, non che due editor lo condividono.

Effetto. Con due editor aperti il registro su disco mescola le righe dei due e ne perde una parte: proprio il file che serve quando un editor si è chiuso da solo.

Correzione proposta. Un file per processo (con il PID nel nome, e un collegamento all'ultimo), oppure l'apertura in append con un blocco sul file, che dica al secondo editor che il registro è già in uso.

7 Lavoro in background

BKG-2 — Un lavoro dell'editor che va in panic sparisce per tutta la sessione, e lo dice solo il registro

Gravità	BASSA
Dove	`work/core.rs` (`Core::run_step`: il lavoro che va in panic esce con `Exit::Panicked`), `editor/boot.rs` (`EditorState::new`, che sottomette evidenziatore e prescansione, e `create_global_env`, che sottomette controllo dei file e salvataggio automatico, una volta sola)
Verifica	Riprodotto sulla revisione `3d846f1` con un lavoro Lisp: dopo `(background-call 'boom (lambda () (set-face 'region "#aéb12")))` il registro dice «[BKG] panic: boom retired», il messaggio del panic va su stderr e un worker contatore continua a contare. Per i lavori dell'editor dalla lettura del codice.

Cosa succede. Da quando BKG-1 è risolta, un panic in un passo ritira solo il lavoro che l'ha causato e lo scrive nel registro con il tag `[BKG]`. Per un lavoro Lisp basta. I lavori dell'editor, evidenziatore, prescansione, controllo dei file e salvataggio automatico, sono però sottomessi una volta sola all'avvio, e un lavoro ritirato non torna: dopo un panic dell'evidenziatore la colorazione si ferma fino alla fine della sessione, e l'area messaggi non dice nulla. Un panic che avvenisse con un lock dell'editor in scrittura lo lascerebbe inoltre avvelenato, e il thread dei comandi andrebbe in panic al primo `.expect("write lock on …")`; oggi non si conosce un panic che avvenga sotto un lock.
Effetto. La colorazione, o il salvataggio automatico, smettono di funzionare senza che chi usa l'editor lo sappia.
Correzione proposta. Risottomettere i lavori dell'editor ritirati, con un limite perché un panic che si ripete non diventi un ciclo, e dire nell'area messaggi quale lavoro si è fermato; per i lock, decidere se uno avvelenato si può riprendere (`PoisonError::into_inner`) o deve chiudere l'editor in modo ordinato.

8 Modi, sintassi e colorazione

SIN-1 — Una regione di sintassi che corrisponde alla stringa vuota blocca il thread di background

Gravità	MEDIA
Dove	`primitives/modes.rs` (`add-syntax-region`, che non controlla i pattern), `modes/syntax.rs` (`highlight_line`, che non avanza dopo un'apertura, una chiusura o un escape vuoti)
Verifica	Riprodotto nell'analisi di `08a853d`: con BEGIN `"x*"` ed END `"y*"` un thread restava al 97 % di CPU e la colorazione di un file di una riga non finiva mai; con BEGIN vuoto e NESTABLE la memoria cresceva di circa 90 MB al secondo; con un pattern normale lo stesso file si colorava in 20 ms. Il ciclo di `highlight_line` non è cambiato da allora.

Cosa succede. `highlight_line` avanza la posizione al punto in cui finisce l'evento trovato. Per una regola protegge il caso vuoto (`pos = resume.max(pos + 1)`), per regioni ed escape no. Una regione che si apre e si chiude con due corrispondenze vuote nella stessa posizione si riapre all'infinito; una regione annidabile con un BEGIN vuoto continua a impilare stati.

Come riprodurlo.

```
(add-syntax-region 'rust-mode "x*" "y*" 'comment)
;; poi aprire un file .rs qualsiasi
;; errore di battitura realistico: BEGIN "#*" con END "$"
```

Effetto. Il passo dell'evidenziatore non finisce, e con lui si ferma l'esecutore del WorkManager: prescansione, controllo dei file, salvataggio automatico, worker Lisp e `background-call` restano fermi; i comandi di shell e le ricerche, che hanno un thread proprio, no. Il watchdog lo scrive nel registro («[BKG] stuck step: highlighter running for 1.0 s», poi a 2 s, a 4 s…). Oppure il processo esaurisce la memoria. Le grammatiche fornite con l'editor, comprese quelle nuove di Python, TOML, Bash e C, non hanno il problema; basta però un errore di battitura in una grammatica scritta dall'utente.

Correzione proposta. Rifiutare in `add-syntax-region` i pattern per cui `is_match("")` è vero, e in `highlight_line` forzare l'avanzamento di almeno un carattere dopo un evento vuoto.

SIN-2 — Colorare una riga molto lunga costa un tempo quadratico nella sua lunghezza

Gravità	MEDIA
Dove	`modes/syntax.rs` (`top_level_event`, che riprova ogni regione e ogni regola dalla posizione corrente, e `highlight_line`), `buffer/syntax.rs` (`LINES_PER_TURN`)
Verifica	Misurato nell'analisi di `08a853d` su una riga Rust ripetuta (`let a = "x";`): 65 KB in 0,96 s, 130 KB in 3,7 s, 260 KB in 14,8 s; una riga di 1 MB (un file minificato) richiederebbe circa quattro minuti. `top_level_event` non è cambiata da allora.

Cosa succede. A ogni evento la scansione chiede a ogni pattern la sua prossima corrispondenza a partire dalla posizione corrente. I pattern che non trovano nulla scorrono ogni volta fino alla fine della riga, quindi il lavoro è proporzionale a eventi × lunghezza. Una riga non si può dividere tra due turni, perché `LINES_PER_TURN` conta righe.

Effetto. Mentre l'esecutore del WorkManager colora la riga, prescansione, controllo dei file, salvataggio automatico e worker sono fermi, e il watchdog scrive nel registro un passo lungo e poi bloccato.

Correzione proposta. Ricordare per ogni pattern la prossima corrispondenza trovata e ricalcolarla solo quando la posizione la supera; oltre una certa lunghezza, non colorare la riga.

SIN-4 — `make-mode` su un modo esistente lo sostituisce con uno vuoto

Gravità	BASSA
Dove	`primitives/modes.rs` (`make-mode`), `editor/modes.rs` (`EditorState::set_mode`), `managers/modes.rs` (`Modes::insert`)
Verifica	Dalla lettura del codice.

Cosa succede. Rivalutare il file di un modo (per esempio dopo una modifica) azzera la sua mappa dei tasti, i suoi hook, la grammatica e la tabella di sintassi, compresi quelli che altri moduli avevano aggiunto (come l'hook di `electric-pair`). La docstring di `make-mode` e i limiti della pagina `writing-modes` ora lo dicono.
Correzione proposta. Se il modo esiste già, restituirlo senza modificarlo, oppure svuotare solo grammatica e tabella.

SIN-5 — Il registro delle facce cresce per sempre e l'indice è a 16 bit

Gravità	BASSA
Dove	`ui/faces.rs` (`Face::intern`, `Face::all`)
Verifica	Dalla lettura del codice.

Cosa succede. Ogni nome di faccia mai usato resta registrato. `(names.len() - 1) as u16` si avvolge dopo 65 536 nomi distinti, e un nome nuovo prenderebbe l'indice di uno vecchio. I nomi arrivano anche da Lisp mentre l'editor gira (`set-face`, `face-style`, le facce di overlay e testo virtuale), quindi il commento che dice il registro limitato dalla configurazione è ottimista.
Correzione proposta. Un controllo con errore oltre `u16::MAX`, o un indice a 32 bit.

SIN-6 — `forward-sexp` dall'interno di un simbolo salta alla fine dell'espressione successiva

Gravità	MEDIA
Dove	`modes/sexp.rs` (`forward`, `Scan::run_to`, `Scan::step`); la usano `forward-sexp`, `kill-sexp` e `backward-sexp` con un conteggio negativo
Verifica	Riprodotto sulla revisione `3d846f1`: in `(foo bar) (baz)`, da 2 (dentro `foo`) `forward-sexp` porta a 8 (fine di `bar`) invece che a 4; da 13 (dentro `baz`) non si muove invece di arrivare a 14.

Cosa succede. `forward` porta la scansione fino a `from` con `run_to`, poi conta le espressioni completate da lì in avanti. Ma un passo della scansione consuma un intero simbolo: se `from` cade dentro un simbolo, `run_to` lo completa (e lo scarta) prima di restituire il controllo, e il primo completamento contato è quello dell'espressione dopo. Sul simbolo finale di una lista non resta nulla da contare a quel livello e il punto non si muove.
Effetto. `C-M-f` e `kill-sexp` (che usa la stessa funzione) dall'interno di una parola si comportano in modo diverso da Emacs: il primo salta un'espressione, il secondo cancella anche l'espressione successiva.
Correzione proposta. In `run_to` fermarsi prima di un passo che supererebbe il limite, oppure in `forward` controllare se l'ultimo completamento di `run_to` finisce dopo `from` e contarlo.

9 Ricerca, sostituzione e liste di risultati

RIC-1 — Una sostituzione con un'espressione che può essere vuota mette tutte le sostituzioni nello stesso punto

Gravità	MEDIA
Dove	`text/search.rs` (`Replace::accept`), `editor/replace.rs` (`EditorState::replace_rest`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(replace-regexp "x*" "-")` su `abc` dà `-----abc` invece di `-a-b-c-`, e dice «Replaced 4 occurrences» dopo averne fatte cinque.

Cosa succede. Dopo una sostituzione di una corrispondenza vuota, la ricerca riprende da `found.start + new_len`, cioè subito dopo il testo appena inserito e **prima** del carattere successivo. Lì trova un'altra corrispondenza vuota, e così via. Si ferma solo per il tetto di sicurezza; il ciclo lo controlla dopo aver già sostituito, quindi le sostituzioni sono `len + 2` e il messaggio ne conta una di meno.
Correzione proposta. Per una corrispondenza vuota riprendere da `found.start + new_len + 1`, scavalcando un carattere, come fa Emacs; e contare le sostituzioni fatte.

RIC-2 — La ricerca all'indietro con un'espressione regolare costa un tempo quadratico

Gravità	MEDIA
Dove	`text/search.rs` (`Pattern::search_backward`, ramo `Regex`), usata da `feature/isearch.rs` (`scan`)
Verifica	Misurato nell'analisi di `08a853d` con `C-M-r` dalla fine di un file `ab ab ab …`, per il solo tasto `b`: 15 KB in 0,24 s, 30 KB in 0,99 s, 60 KB in 3,59 s. `text/search.rs` non è cambiato da allora.

Cosa succede. Il motore delle espressioni cerca solo in avanti, quindi `search_backward` chiama `search_forward` in un ciclo e tiene l'ultima corrispondenza trovata. Ma ogni chiamata a `search_forward` copia l'intero buffer in una `String` e converte gli offset con una scansione lineare. È lo stesso costo quadratico che il commento sopra `Pattern::scan` dice di evitare.
Effetto. Su un file di qualche centinaio di KB con molte corrispondenze, ogni tasto premuto durante `C-M-r` blocca l'editor per minuti.
Correzione proposta. Una sola copia del testo e un solo passaggio con `captures_iter` che tiene l'ultima corrispondenza che finisce prima del limite.

RIC-3 — Ogni sostituzione è un gruppo di annullamento a sé

Gravità	BASSA
Dove	`editor/replace.rs` (`EditorState::replace_this`, che chiama `undo.boundary()` a ogni sostituzione)
Verifica	Riprodotto sulla revisione `3d846f1`: dopo `(replace-string "a" "b")` su `aaa`, che dice «Replaced 3 occurrences», un `undo` torna a `bba` e non ad `aaa`.

Cosa succede. Il confine di annullamento serve a `query-replace` per tornare indietro di una sostituzione, ma viene messo anche nelle sostituzioni non interattive.
Correzione proposta. Nessun confine tra le sostituzioni di `replace-string` e `replace-regexp`; un confine solo nel percorso interattivo.

RIC-4 — Limiti della ricerca confinata a una regione e delle scansioni su righe lunghe

Gravità	BASSA
Dove	`text/search.rs` (`Pattern::search_forward` con LIMIT, `Pattern::search_backward`, `Cursor::column`, `Pattern::scan`)
Verifica	Riprodotto sulla revisione `3d846f1`: su `aaaa` con la regione `[0, 2)`, `(replace-regexp "a+" "X")` risponde «No match for a+». Il costo della colonna viene dalla lettura del codice.

Cosa succede. Se la prima corrispondenza dopo il punto finisce oltre il limite, viene scartata invece di cercarne una più corta: `a+` su `aaaa` con la regione `[0, 2)` non trova nulla, anche se la regione contiene `aa`. Lo stesso vale per la ricerca incrementale all'indietro con un'espressione, che si ferma alla prima corrispondenza che supera il punto. Nella scansione completa (occur, grep), `Cursor::column` ricalcola i caratteri dall'inizio della riga per ogni corrispondenza, quindi una riga lunghissima con molte corrispondenze costa un tempo quadratico; grep però gira ora su un thread suo, e solo occur blocca l'editor.

Correzione proposta. Cercare nella sola fetta della regione; tenere la colonna in modo incrementale come già si fa per la riga.

RIC-5 — I risultati di occur, grep e compile puntano a righe che si spostano

Gravità	BASSA
Dove	`text/results.rs` (`Entry`, `Results::step`), `primitives/results.rs` (`visit`, `visit_index`, `refresh`)
Verifica	Riprodotto sulla revisione `3d846f1`: in `x`, `foo`, `bar`, dopo `(occur--scan "foo")` e una riga aggiunta in cima, `next-error` porta il punto all'inizio della riga di `x`.

Cosa succede. Ogni risultato conserva riga, colonna e offset come numeri, e `visit` si sposta per riga e colonna. Dopo una modifica che aggiunge o toglie righe sopra un risultato, `next-error` porta alla riga sbagliata; Emacs usa marcatori che seguono il testo. Il solo modo di aggiornare una lista, `results-refresh`, la ricostruisce da capo, e lo usa solo `compile` con l'uscita del compilatore.

Correzione proposta. Marcatori nei buffer aperti, aggiornati dalle due porte delle modifiche.

RIC-7 — Una lista di `M-s o` e una di `M-s g` aprono due strisce, una sopra l'altra

Gravità	BASSA
Dove	`editor/results.rs` (`display_results`, che cerca la striscia da riusare), `primitives/results.rs` (`occur--scan` e `grep--scan`, che si presentano come produttori diversi)
Verifica	Riprodotto nell'editor sulla revisione `3d846f1`: una lista di `M-s o` e poi una di `M-s g` lasciano tre finestre, la lista di grep in una striscia nuova sopra quella di occur. Nessun test prova una lista di un tipo seguita da una dell'altro.

Cosa succede. `display_results` riusa la striscia in basso solo se la lista che vi è mostrata viene dallo stesso produttore, e occur e grep sono due produttori, `"occur"` e `"grep"`, anche se le loro liste sono entrambe in `occur-mode`. Prima di `3d846f1` una seconda lista, di qualunque dei due comandi, prendeva la striscia della prima.

Effetto. Chi cerca prima nel buffer e poi nella directory si ritrova con lo schermo diviso in tre, e deve chiudere la striscia in più con `q`. La pagina `searching` lo dice tra i limiti.

Correzione proposta. Se le due liste devono condividere la striscia, scegliere la striscia da riusare per tipo di lista (il modo del buffer) invece che per produttore, oppure dare a occur e a grep lo stesso produttore; in entrambi i casi un test con una lista di `M-s o` seguita da una di `M-s g`, e la riga tolta dai limiti della pagina.

10 Comandi di shell

SHL-4 — Il nome di una pagina di manuale viene passato alla shell senza virgolette

Gravità	BASSA
Dove	`core/lisp/manpage.lisp` (`manpage--system-page`, che passa a `shell-command-to-string` il comando `man` con il nome dentro; `manpage--rsedit-page`, che costruisce il percorso della pagina concatenando il nome)
Verifica	Riprodotto sulla revisione `3d846f1`: `(manpage--system-page "printf;touch FILE")` crea FILE.

Cosa succede. `(format "man -w %s 2>/dev/null" name)` passa il nome così com'è a `/bin/sh -c`. Un nome con spazi o metacaratteri viene interpretato dalla shell: `printf(3)` dà un errore di sintassi, che si presenta come «No manual entry», e `a;b` esegue due comandi. Il nome arriva senza filtri da `M-x manpage` e da `C-h m`; da `manpage-at-point` (e da `K` in vim) è limitato ai caratteri di un simbolo, che comprendono `*`, espanso dalla shell. Anche il percorso della pagina interna viene costruito concatenando il nome, quindi `../` esce dalla directory dei dati.

Correzione proposta. Racchiudere il nome tra apici singoli, con gli eventuali apici interni preceduti da escape, oppure rifiutare i nomi che non sono `[A-Za-z0-9._+-]+`.

11 Disegno e terminale

TUI-1 — I caratteri di controllo del testo arrivano al terminale: un file può comandarlo

Gravità	ALTA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Layout::visible_text`: copiano i caratteri senza filtrarli), `src/tui.rs` (`draw_clipped_row`, che li stampa con `Print`)
Verifica	Riprodotto nell'analisi di `08a853d`: in un file che contiene `ESC ] 52 ; c ; aGk= ESC \` la riga della fotografia conteneva la sequenza intatta, che la TUI scrive sul terminale. `ui/layout.rs` non è cambiato da allora, e `src/tui.rs` solo nell'attesa degli eventi.

Cosa succede. Tra il buffer e il terminale nessuno sostituisce i caratteri di controllo (ESC, BEL, CR, BS, DEL, i controlli C1). Il terminale li esegue. Passa dalla stessa strada l'uscita dei comandi, che da `02ea0e7` arriva decodificata dai byte così com'è. La GUI (`src/gui.rs`) salta invece i caratteri di controllo.

Effetto. Aprire un file, guardare l'output di un comando, un risultato di grep o una pagina di manuale che contengono sequenze di escape fa eseguire quelle sequenze al terminale. OSC 52 riscrive gli appunti di sistema: l'utente incollerà poi in una shell un comando scelto da chi ha preparato il file. OSC 0 cambia il titolo della finestra; le sequenze CSI spostano il cursore, cancellano lo schermo o cambiano i colori. Anche senza malizia, un `\r` o un `\b` in un file rovinano la visualizzazione.

Correzione proposta. Al momento del disegno, sostituire i controlli (U+0000–U+001F tranne il tab, U+007F, U+0080–U+009F) con una rappresentazione visibile, come `^[` in Emacs o il carattere `␛`, contandone la larghezza nella mappa delle colonne.

TUI-2 — Un panic o un errore di I/O lasciano il terminale in raw mode

Gravità	ALTA
Dove	`src/tui.rs` (`tui_main`), `src/main.rs` (`main`: nessun `panic::set_hook`, nessun `catch_unwind`; la guardia `ShutDownOnExit` ferma solo il lavoro in background)
Verifica	Dalla lettura del codice: nessun `set_hook`, e `catch_unwind` solo nel WorkManager. Un panic che lo innesca oggi è quello di TUI-5, riprodotto sulla revisione `3d846f1`.

Cosa succede. `tui_main` attiva il raw mode, il bracketed paste e, a richiesta, la cattura del mouse, e li disattiva solo sul percorso di uscita normale. Un panic nel thread dei comandi risale fuori da `main` senza ripristinarli; ogni `?` su un errore di I/O nel ciclo (`terminal::size`, `render_frame`, la cattura del mouse, un errore del thread che legge il terminale) esce da `tui_main` saltando il ripristino. Anche un panic che il WorkManager intercetta, in un lavoro di background, scrive il suo messaggio su stderr, sopra lo schermo.

Effetto. Dopo un crash la shell non mostra ciò che si digita, Invio non funziona e i clic del mouse stampano sequenze di escape. L'utente deve digitare `reset` alla cieca. Il messaggio del panic stesso viene stampato «a scalini», in raw mode.

Correzione proposta. Una struttura di guardia il cui `Drop` ripristina il terminale, più un `panic::set_hook` che ripristina prima di stampare il messaggio.

TUI-3 — Le tabulazioni non vengono espanse

Gravità	MEDIA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Row::to_screen`, `Row::to_buffer`: una colonna del buffer è una colonna dello schermo), `src/tui.rs` (`render_to`, `draw_clipped_row`)
Verifica	Riprodotto nell'analisi di `08a853d`: la riga della fotografia conteneva il carattere `\t`. `ui/layout.rs` non è cambiato da allora.

Cosa succede. L'editor conta un tab come una colonna. Il terminale invece lo espande fino alla tabulazione successiva, quindi il resto della riga viene disegnato più a destra di dove l'editor crede. Il commento in cima a `text/rectangle.rs` («a tab is drawn in one cell») descrive il modello interno, non ciò che accade sullo schermo. Esiste ora un'impostazione `tab-width`, nel modulo `indent`, ma serve solo al rientro: il disegno non la legge.

Effetto. In un Makefile, in un file Go o in qualsiasi sorgente indentato con tab, il cursore appare nel punto sbagliato, i colori cadono sulle parole sbagliate e il resto della riga precedente può restare visibile.

Correzione proposta. Espandere i tab in spazi nella composizione delle righe (`compose_row`), con una larghezza configurabile, e usare la stessa mappa per il cursore e per i clic del mouse.

TUI-4 — I caratteri larghi contano come una colonna

Gravità	MEDIA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Row::to_screen`, `Row::to_buffer`), `src/tui.rs` (`render_to`, che riempie le righe contando i caratteri)
Verifica	Dalla lettura del codice: le larghezze sono tutte `chars().count()`, e nessuna dipendenza calcola la larghezza di visualizzazione (`unicode-width`).

Cosa succede. Un ideogramma o un'emoji occupa due celle del terminale, ma l'editor ne conta una.
Effetto. Come per i tab: cursore, evidenziazioni, ritaglio e riempimento delle righe sfasati nei testi in cinese, giapponese, coreano o con emoji.
Correzione proposta. Calcolare la larghezza con `unicode-width` nella stessa mappa di TUI-3.

TUI-5 — Un colore esadecimale con caratteri non ASCII manda l'editor in panic

Gravità	MEDIA
Dove	`ui/faces.rs` (`Color::parse`), chiamata da `primitives/theme.rs` (`colour_arg`, dietro `set-face` e `define-face`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(set-face 'region "#aéb12")` va in panic con «end byte index 2 is not a char boundary».

Cosa succede. `Color::parse` controlla la lunghezza in byte e poi taglia la stringa con `&hex[0..2]`. Con un carattere di due byte il taglio cade a metà del carattere.
Effetto. Chiusura dell'editor, con il terminale in raw mode (TUI-2). Il caso si presenta anche con un file di tema sbagliato. In un lavoro di background il panic ritira solo quel lavoro.
Correzione proposta. Rifiutare i colori che non sono ASCII (`hex.is_ascii()`) prima di tagliare.

TUI-6 — Le finestre flottanti non mostrano colori di sintassi né overlay

Gravità	BASSA
Dove	`editor/frame.rs` (`EditorState::snapshot`, il ciclo sulle finestre flottanti), a confronto con `ui/windows.rs` (`LayoutNode::compute_tiled_views`)
Verifica	Dalla lettura del codice: per una finestra flottante si raccolgono solo la regione e il testo virtuale.

Cosa succede. Un buffer mostrato in una finestra flottante appare senza colori e senza le evidenziazioni degli overlay; lo stesso buffer in una finestra affiancata le ha.
Correzione proposta. Usare per le finestre flottanti la stessa raccolta delle evidenziazioni delle finestre affiancate.

TUI-7 — `%p` nella riga di stato dice `All` solo per un buffer di una riga

Gravità	BASSA
Dove	`ui/windows.rs` (`position_in_buffer`, ed `expand_mode_line`, che non riceve l'altezza della finestra)
Verifica	Dalla lettura del codice.

Cosa succede. Il commento dice «`All` quando l'intero buffer sta in una schermata», ma il codice verifica `lines <= 1`. Il test `the_position_reads_as_emacs_words_it` fissa il comportamento di oggi.
Correzione proposta. Passare l'altezza della finestra e confrontare con quella.

TUI-8 — La TUI non usa lo schermo alternativo

Gravità	BASSA
Dove	`src/tui.rs` (`tui_main`, `render_to`)
Verifica	Dalla lettura del codice: nessun `EnterAlternateScreen`.

Cosa succede. L'editor disegna sullo schermo principale del terminale e all'uscita lo cancella con `Clear(All)`.
Effetto. Il contenuto che il terminale mostrava prima di avviare l'editor, per esempio l'output di un comando, va perso.

Correzione proposta. `EnterAlternateScreen` all'avvio e `LeaveAlternateScreen` all'uscita, nella stessa guardia di TUI-2.

12 Documentazione, commenti, codice morto, build e test

DOC-1 — Commenti e docstring che descrivono un comportamento che non c'è più

> **Gravità** BASSA
> **Dove** vedi l'elenco
> **Verifica** Dalla lettura del codice.

Cosa succede.

- `feature/minibuffer.rs` (il commento del modulo) e `src/tui.rs` (`render_to`, il commento sull'area messaggi): il minibuffer «docked to the last few lines of the frame», mentre il prompt predefinito ora è centrato.
- `editor/buffers.rs` (`EditorState::close_buffer`): «this editor does not yet ask before that kill», mentre `kill-buffer` ora chiede conferma.
- `buffer/undo.rs` (`UndoHistory::limit`): «How many groups this history keeps», mentre il limite è in byte di testo cancellato.
- `text/rectangle.rs` (il commento del modulo): «a tab is drawn in one cell» (TUI-3).
- `primitives/shell.rs` (`shell-command-start`): il commento «Only as the editor shuts down» e il messaggio «The editor's background work has shut down» valgono anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta.

Le voci su `primitives/workers.rs` e su `shell-command-to-string` non valgono più: il codice e la docstring sono stati riscritti (`6de16a1`, `4f05134`). Le tre sui commenti della libreria di base dell'interprete sono nel rapporto di risp, dentro le schede INT-9, INT-16 e INT-21.

Correzione proposta. Aggiornarli insieme alle correzioni dei problemi a cui rimandano.

DOC-2 — Codice morto e trappole latenti

> **Gravità** BASSA
> **Dove** `buffer/mod.rs` (il campo `Buffer::local_keymap`), `editor/mod.rs` (`impl Debug for EditorState`, e accanto `impl PartialEq for EditorState`)
> **Verifica** Dalla lettura del codice.

Cosa succede. `Buffer::local_keymap` viene inizializzato ma mai letto; la mappa locale che funziona, quella di `set-local-keymap`, sta nei dati del buffer. `Debug` per `EditorState` è un `todo!()` e `PartialEq` un `unreachable!()`: li richiede il trait `LispContext` di risp. Un `dbg!`, un `{:?}` o un `assert_eq!` fallito su una struttura che contiene uno stato dell'editor va in panic invece di stampare; oggi nessun tipo di risp contiene un contesto, e la trappola resta latente.

Correzione proposta. Togliere il campo; un `Debug` che stampi almeno i nomi dei buffer, e un `PartialEq` per identità.

DOC-5 — Il codice delle liste di risultati lascia un metodo morto, due `if` da unire e righe da formattare

> **Gravità** BASSA
> **Dove** `editor/buffers.rs` (`free_buffer_name`), `editor/results.rs` (`select_results_target`, `quit_results`), e i cinque file toccati da `3d846f1`: `editor/results.rs`, `editor/settings.rs`, `editor/boot.rs`, `primitives/results.rs`, `primitives/shell.rs`
> **Verifica** Riprodotto sulla revisione `3d846f1`: ogni `cargo build` avvisa che `free_buffer_name` non è mai usato; `cargo clippy` dà tre avvisi in più della revisione precedente, `5f79477`; `cargo fmt --check` trova 41 blocchi da riformattare in quei cinque file, dove prima ne trovava 3, tutti in `src/gui.rs`.

Cosa succede. Con `3d846f1` le liste di occur si aprono con `prepare_results_buffer`, che sceglie il nome con `free_name_for_work`: se ne va così l'ultima chiamata a `free_buffer_name`, e il metodo resta, con la sua docstring. In `select_results_target` e in `quit_results` un `if let` contiene un solo `if`, che clippy chiede di unire (`collapsible_if`). Il codice nuovo non è passato da `rustfmt`: righe spezzate dove ci starebbero intere, la registrazione di `results-display` e `results-quit` con l'indentazione persa, spazi in fondo a una riga di `primitives/shell.rs`.

Effetto. Nessuno per chi usa l'editor. Un avviso fisso in ogni compilazione nasconde quelli nuovi, e la prossima modifica a questi file, passata da `rustfmt`, porterà con sé decine di righe che non c'entrano.
Correzione proposta. Togliere `free_buffer_name`, unire i due `if`, passare `cargo fmt` sui cinque file, in un commit che non cambia altro.

13 Appendici

13.1 Le schede risolte

Le schede risolte dopo l'analisi, con il commit che le ha risolte. Il testo completo di ciascuna sta nella storia di git di questo file; che cosa è cambiato, e perché, nel messaggio del commit.

ID	Problema	Gravità	Risolto in
AVV-1	L'editor non parte se la directory corrente non è scrivibile	ALTA	`58d0b0d`
AVV-3	Un errore di scrittura del registro manda l'editor in panic	MEDIA	`a20f050`
AVV-4	Il registro in memoria cresce senza limite	BASSA	`a20f050`
BKG-1	Un panic in un lavoro di background uccide per sempre il thread condiviso	ALTA	`6de16a1`
SIN-3	`write-file` che cambia il modo lascia i colori del modo precedente	BASSA	`cf0d8ac`
RIC-6	Alla fine di una ricerca incrementale il mark non è dove la ricerca è cominciata	BASSA	`1aa5cf7`
SHL-1	Un comando che scrive molto su stderr si blocca	MEDIA	`02ea0e7`
SHL-2	Un solo byte non UTF-8 nell'output interrompe il comando	MEDIA	`02ea0e7`
SHL-3	Non c'è modo di fermare un comando in corso	BASSA	`02ea0e7`
DOC-3	Lo script di build presume dove si trova la directory `target`	BASSA	`f742701`
DOC-4	Un test di prestazioni instabile e clippy che fallisce sui test	BASSA	`10598fa`

13.2 Differenze da Emacs che non sono difetti

Alcuni comportamenti differiscono da Emacs per scelta, o comunque senza produrre risultati sbagliati. Sono elencati qui perché chi conosce Emacs non li scambi per errori. Quelle dell'interprete sono nel rapporto di risp.

- Le espressioni regolari usano la sintassi del crate `regex` di Rust, non quella di Emacs. `\(` è una parentesi letterale e i gruppi si scrivono `( )`. Non esistono lookaround né riferimenti all'indietro, ed è proprio ciò che garantisce un tempo lineare.
- Le liste di risultati, le ricerche e le sostituzioni non tornano mai all'inizio da sole: quando la lista finisce lo dicono, come scelta esplicita di progetto.

13.3 Parti esaminate e trovate solide

Nell'analisi queste parti sono state esercitate di proposito sui casi limite senza trovare problemi:

- lo scanner delle espressioni bilanciate (`modes/sexp.rs`): stringhe con escape alla fine del buffer, parentesi sbilanciate in entrambe le direzioni, letterali carattere, lifetime di Rust, stringhe raw, commenti a blocco annidati, commenti che contengono parentesi;
- la ricerca incrementale: parentesi letterali, espressioni non valide, cancellazione, ricerca fallita e ripresa dopo il fallimento;
- kill e yank, compreso `M-y`, e le macro di tastiera (con il limite alla ricorsione delle macro);
- `kill-buffer` di un buffer mostrato in due finestre, la chiusura dell'unica finestra, la chiusura di `*scratch*`;
- la sostituzione `a` → `aa`, che termina;
- i comandi di modifica ai bordi del buffer;
- `delete-file` sui link simbolici a directory, che cancella il link e non il contenuto della directory di destinazione;
- l'uscita con buffer non salvati: dopo un salvataggio fallito il buffer resta segnato come modificato e l'editor chiede conferma prima di uscire.